Array Notes

By Divyanshu Shukla

- 12 Easy Level Questions
- 8 Medium Level Questions
- 9 Hard Level Questions

Questions based on Take U Forward's Strivers A2Z DSA Course/Sheet

Link of Sheet - https://takeuforward.org/strivers-a2z-dsa-course/strivers-a2z-dsa-course-sheet-2/

Array [Easy Questions]

Date __/__/____
Page _____

Q1 Largest element in an array.

- We will initialize max = Integer.MIN_VALUE;
- Traverse the array and update max on finding a larger number.

```
int max = Integer.MIN_VALUE;
for (int i=0; i<a.length; i++)
    {
        max = Math.max (max, a[i]);
    }
return max;
```

Q2 Find kth smallest / largest element in an array.

- Sort the array in acending / decending order in accordance with the question
- return a[k-1] element.

```
Arrays.sort (nums);
return nums[k-1];
```

Q3 Check if the array is sorted.

- Compare each element with the element before, sorted arrays always have larger or equal element after every element.

```
for (int i=1, i<a.length; i++)
    if (a[i-1] > a[i])
        return false;
```

Page

Q4 Left rotate an array by K places.

- Reverse the array from [0 to k-1] index
- Reverse the array from [k to n-1] index
- Reverse whole array [0 to n-1] index.

Q5 Right rotate an array by K places.

- Reverse whole array from [0 to n-1] index.
- Reverse the array from [0 to k-1] index.
- Reverse the array from [k to n-1] index.

```
public void reverse(int[] a, int i, int j)
    {
      while (i<j)
        {
          int temp = a[i];
          a[i] = a[j];
          a[j] = temp;
          i++;
          j--;
        }
    }
```

Date __/__/__
Page _____

Q6 Move zeros to the end of array.

- Use snowball technique.

```
Snowball = 0 ;
for (int i=0 ; i< a. length ; i++)
    {
        if ( a[i] == =0)
            snowball ++ ;
        else if ( snowball >0 )
            {
                int temp = a[i] ;
                a[i] = 0 ;
                a[i-d] = temp ;
            }
    }
```

Q7 Union of 2 sorted arrays

- We will use merge sort technique.
- Use 2 pointers to add.
- Check for the element present before each instance.

```
List < Integer > ans = new ArrayList <> ( ) ;
Int i =0 , j=0 ;
while ( i<n && j<m)
    {
        if ( a1[i] <= a2[j])
            {
```

```
if ( ans.size() ==0 || ans.get(ans.size()-1) != a1[i])
  {
     ans.add(a1[i]);
  }
     i++;
  }
else {
    if( ans.size()==0 || ans.get(ans.size()-1) != a2[j])
      {
        ans.add(a2[j]);
      }
    j++;
  }
}
while (i<n)
   {
     if (ans.get(ans.size()-1) != a1[i])
        ans.add (a1[i]);
        i++;
   }
while (j<m)
   {
     if (ans.get(ans.size()-1) != a2[j])
        ans.add (a2[j]);
        j++;
   }
```

Date __/__/__
Page

Q8 Intersection of 2 sorted arrays.

- Use 2 pointer approach
- check if the element is present in both array then add it into answer.

```
List < Integer > ans = new ArrayList <>();
int i=0 ; j=0;
    while ( i < m  && j < m )
        {
            if ( a[i] < B[j] )
                i++;
            else if ( a[i] > B[j] )
                j++;
            else {
                ans.add ( a[i]);
                i++;
                j++;
            }
    }
```

Q9 Find missing number in an array.

- Calculate the sum of all elements and store it in sum2.
- Calculate the sum of min to length.
- Subtract sum1 - sum2 for answer.

Date __/__/____
Page _____

Q10 Find the number which appears once and all other numbers twice.

- Take an element as 0.
- XOR all elements of array from that element.
- Return the element.

```
int ans = 0;
for (int i=0; i<a.length; i++)
    ans = ans ^ a[i];
return ans;
```

Q11 Longest subarray with given sum

- Store the sum and index in a hashmap
- If the sum - k is found in map, the length of sub array is i - index at sum-k.
- Store the length and update for max length.

```
HashMap <Integer, Integer> map = new HashMap<>();
int len = 0;
int sum = 0;
for (int i=0; i<a.length; i++)
{
    sum = sum + a[i];
    if (sum == k)
    len = Math.max (len, i+1);
    int rem = sum - k;
```

```
if (map.containsKey (rem))
    len = Math.max ( len, i - map.get (rem));
if (! map. containsKey (sum))
    map.put ( sum, i);
  3
  return len;
```

Q12 Sort arrays of 0, 1, and 2.

- Use 3 pointer approach (Dutch national flag algorithm)

```
int i = 0;
int j = 0;
int k = n-1;
while ( j <= k)
    {
      if (a[j] == 0)
        {
          swap ( a[i], a[j]);
          i++;
          j++;
        }
      else if ( a[j] == 1)
          j++;
      else {
          swap (a[j], a[k]);
          k--;
        }
}
```

Array [Medium]

Date __/__/____
Page _____

Q1 Majority element in array [>N/2]

- Use moose voting algorithm
- initialize count and ele as 0, for first ele = a[i]
  and count = count + 1
- If the next ele = ele, then reduce count
- If count = 0, then put next element as ele.

```
int count = 0;
int ele = 0;
for (int i=0; i< a.length ; i++)
    {
if (count ==0)
    {
    count = 1;
    ele = a[i];
    }
else if ( ele == a[i])
    count ++;
else
    count --;
}
return ele
// note : ele might be the majority element
    but always check for >N/2 condition.
```

Date __/__/__
Page

Q2 Maximum Subarray Sum

- Use Kadane's algorithm
- add sum at every iteration of array.
- If at anytime sum became less than 0, then set sum to 0.

```
int max = Integer.MIN_VALUE;
int sum = 0;
for(int i=0; i<a.length; i++)
    {
        sum = sum + a[i];
        max = Math.max(sum, max);
        if (sum<0)
            sum = 0;
    }
return max;
```

Q3 Print next permutation.

- Find the first element from end of the array which is not increasing order.
- If no such element it found, then you are at the last permutation, so the answer will be the first permutation, hence reverse the array and that reversed array will be the answer.
- If the element is found swap it with the first largest element from the back of array.
- Reverse the array from (ele+1, to n-1).

```
int index = -1;
for( int i = n-2 ; i>=0 ; i--)
    {
        if ( nums[i] < num[i+1])
            {
                index = i;
                break;
            }
    }
if (index == -1) {
    reverse ( 0, n-1);
    return;
}
for(int i = n-1; i>=0; i--)
    {
        if ( nums[i] > nums[index]) {
            swap ( nums[i], nums[index])
            break;
        }
    reverse ( index+1, n-1);
```

Q4 Leaders in an array

- last element is always a leader. and an element after which no element present is larger is also a leader.
- Start iterating from second last, if the element is larger than maximum till then, then it is also a leader.

```
List < Integer > ans = new ArrayList <>();
int max = a[n-1];
ans.add(a[n-1]);
for( int i = n-1; i>=0; i--)
    {
if (max < a[i])
    {
    max = a[i];
    ans.add(a[i]);
    }
}
return ans;
```

Qs Rotate Matrix by 90° to clockwise.

- Transpose the matrix
- Reverse each row of the matrix.

Q6 Rotate a matrix by 90° to anti-clockwise

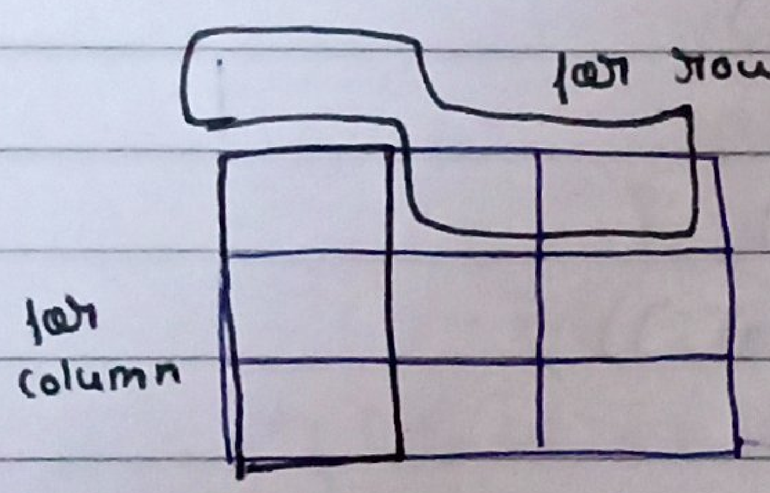
- Reverse each row of the matrix
- Transpose the matrix

```
for(int i=0; i<a.length; i++)
    for( int j=i; j<a.length; i++)
        swap( a[i][j], a[j][i])
    // code for transpose.
```

Date __/__/____
Page _____

```
// code for reversing each row
for (int i=0; i<a.length; i++)
    for (int j=0; j<a.length/2; j++)
        swap(a[i][j], a[i][a.length-1-j]);
```

Q7 Set Matrixes Zero.



- Take first column to store zeros present in each row
- Take first row to store zeros present in each column.
- Use a variable to store first element of a row as 1.
- Starting applying zeros from reverse.

```
int extra = 1;
for (int i=0; i<rows; i++)
  {
    if (matrix[i][0] == 0)
      extra = 0;
    for (int j=1; j<cols; j++)
      {
        if (matrix[i][j] == 0)
          {
            matrix[i][0] = 0;
            matrix[0][j] = 0;
          }
      }
  }
```

Date __/__/__
Page

```
for(int i = rows-1 ; i >= 0 ; i--)
  {
    for ( int j = cols-1 ; j >= 1 ; j--)
      {
        if ( matrix[i][0] == 0 || matrix[0][j] == 0)
          matrix[i][j] = 0;
      }
    if ( extra == 0)
      matrix[i][0] = 0;
  }
```

Qs Find the number of subarrays with sum k.

- Use hashmap to store sum and no. of times sum occur.
- Use prefix sum approach to solve.
- Always put (0,1) in map first. because some subarrays can be of sum 0 and still get added.

```
HashMap < Integer, Integer > map = new HashMap <>();
map.put(0,1);
int sum = 0;
int ans = 0;
for (int i=0; i< a.length ; i++) {
    sum = sum + a[i];
    int rem = sum - k ;
    if ( map.containsKey(rem))
      ans = ans + map.get(rem);
    map.put( sum, map.getOrDefault(sum, 0)+1);
}
return ans;
```

Date __/__/__
Page

Array [Hard]

Q1 Pascal's Triangle

- To find the nth term in the rth ~~row~~ column the formula is -

$$^{row-1}C_{column-1}$$

1) Calculating element

```
long ans = 1;
for(int i = 0; i < r; i++) {
    ans = ans * (n-i);
    ans = ans / (i+1);
}
return ans;
```

2) Finding nth row

```
long ans = 1;
List<Integer> help = new ArrayList<>();
help.add(1);
for(int i=1; i<=n; i++) {
    ans = ans * (n-i);
    ans = ans / i;
    help.add(ans);
}
return help;
```

Date
Page

Q2 Majority element [n/3 times]

```
int num1 = -1;
int num2 = -1;
int c1 = 0;
int c2 = 0;
for( int i=0; i< a.length; i++)
    {
        if (a[i] == num1)
            c1++;
        else if (a[i] == num2)
            c2++;
        else if (c1 ==0)
            {
            c1 = 1;
            num1 = a[i];
            }
        else if (c2 ==0)
            {
            num2 = a[i];
            c2 = 1;
            }
        else
            {
            c1--;
            c2--;
            }
    }
```

- num1 and num2 can be answer, always check.

Page

Q3 Merge overlapping sub intervals.

```
int start = a[0][0];
int end = a[0][1];
for (int i = 0; i < a.length ; i++)
    {
    s = a[i][0];
    e = a[i][1];
    if (s <= end)
        end = Math.max(e, end);
    else {
            ans.add (start, end);
            start = s;
            end = e;
        }
    }
```

Q4 3 sum.

- Use 3 pointers.
- One pointer should be fixed and other two should move from left to right and right to left.
- Sort the array at first.

```
List<List<Integer>> ans = new ArrayList<>();
Arrays.sort(a);
for(int i=0; i<a.length-2; i++)
    {
```

Date __/__/____
Page _____

```
if (i>0 && (a[i] == a[i-1]))
    continue;
int j= i+1;
int k = a.length-1;
while (j<k)
  {
   int sum = a[i] + a[j] + a[k];
   if (sum>0)
      k--;
   else if (sum<0)
      j++;
   else {
   ans.add(Arrays.asList(num[i], a[j], a[k]));
   j++;
   k--;
   while (j<k && nums(a[j] == a[j-1]))
      j++;
   while (j<k && (a[k] == a[k+1]))
      k--;
  }
}
return ans;
```

Page

Qs 4 Sum

- Use 4 pointer approach
- Sort the array
- Never take dublicate elements.

```
List < List < Integer >> ans = new ArrayList <>();
Arrays.sort(a);
for( int i = 0; i < a.length -3; i++)
    {
    if( i>0 && (a[i] == a[i-1]))
        continue;
    for(int j= i+1; j < a.length-2; j++)
        {
        if( j> i+1 && (a[j] == a[j-1]))
        continue;
        int k = j+1;
        int l = a.length -1;
        while( k < l)
          {
        int sum = a[i] + a[j] + a[k] + a[l];
        if (sum > target)
            l--;
        else if ( sum < target)
            k++;
        else {
            ans.add(a[i], a[j], a[k], a[l]);
            k++;
            l--;
```

Date __/__/____
Page _____

```
while (k<e && (a[k] == a[k-1]))
        k++;
while (k<e && (a[e] == a[e+1]))
        e--;
    }
    }
    }
return ans;
```

Q6 Merge 2 sorted arrays without extra space.

1) When extra space is provided at the end of one array.

[1,2,3,0,0,0] [1,7,8]

- Take 3 points, i for last index of 1st array elements, j for last index of 2nd array and k for last index of first array.
- Start filling from the back checking larger number.

```
int i = m-1;
int j = n-1;
int k = m+n-1;
while (j>=0)
{
if ( i>=0 && (nums[i] > nums[j]))
 nums1[k--] = nums1[i--];
else
 nums1[k--] = nums2[j--];
}
```

2) When no extra space is provided and answer is merger of two given arrays.

- Put one pointer at the end of first array and another at the front of second array.
- Swap until the element at second array becomes larger than element of first array.
- Sort both arrays.

```
int i = n-1;
int j = 0;
while (i >= 0 && j < m)
{
if (nums[i] > num[j])
{
swap (nums[i], nums[j]);
i--;
j++;
}
else
break;
}
Arrays.sort(nums1);
Arrays.sort(nums2);
```

Page

Q7 Maximum product sub array.

- Use kadane's algorithm

```
int prod1 = a[0];
int prod2 = a[0];
int ans = a[0];
for( int i = 1; i<a.length ; i++)
  {
   int temp = Math.max (a[i], Math.max (prod1*a[i], prod2*a[i]))
prod2 = Math.min(a[i], Math.min (prod1*a[i], prod2*a[i])),
 prod1 = temp;
 ans = math.max (ans, prod1);
}
return ans;
```

Q8 Count inversions

- Use mergeSort algorithm
- Always update the array after each inversions.
- Use the sorted property of sub-arrays.
- Count is always mid - left + 1

```
public static long mergeSort (int[] a, int low, int high)
  {
int count = 0;
if ( low >= high)
  return count;
int mid = (low + high) / 2;
```

```
count = count + mergesort(a, low, mid);
count = count + mergesort(a, mid+1, high);
count = count + merge(a, low, high, mid);
return count;
}

public static long merge(int a[], int low, int mid, int high)
    {
List<Integer> ans = new ArrayList<>();
int left = low;
int right = mid+1;
int count = 0;
while (left <= mid && right <= high)
    {
if(a[left] <= a[right])
  ans.add(a[left++]);
else {
    count = count + mid - left + 1;
    ans.add(a[right++]);
  }
}
while (left <= mid)
  ans.add(a[left++]);
while (right <= high)
  ans.add(a[right++]);
for(int i = low; i <= high; i++)
   a[i] = ans.get(i - low);
return count;
}
```

Page

Qs Reverse Pairs

- Use merge sort algorithm
- Always update the array at each instance.
- count is always equal to right - mid + 1;

```
public int mergesort(int[] a, int low, int high)
    {
        int count = 0;
        if( low >= high)
            return count;
        int mid = (low + high) / 2;
        count = count + mergesort(a, low, mid);
        count = count + mergesort(a, mid+1, high);
        count = count + merge(a, low, mid, high);
        return count;
    }
public int merge(int[]a, int low, int mid, int high)
    {
        int count = 0;
        int right = mid+1;
        for( int i = low; i <= mid; i++)
            {
            while( right <= high &&
                    a[i] > (2*(long) a[right]))
            right++;
            count = count + right - mid +1;
    }
List<Integer> ans = new ArrayList<>();
```

```
int left = low;
    right = mid + 1;
while (left <= mid && right <= high)
    {
        if (a[left] <= a[right])
            ans.add(a[left++]);
        else
            ans.add(a[right++]);
    }
while (left <= mid)
    ans.add(a[left++]);
while (right <= high)
    ans.add(a[right++]);
for (int i=low; i<= high; i++)
    a[i] = ans.get(i-low);
return count;
}
```